

Day 5 - Flexbox

Day 5 — Flexbox

Objectives

- Build the core mental model: a flex container and its flex items, and the main axis vs. cross axis
- Use `justify-content` and `align-items` to control alignment along each axis
- Use `flex-grow`, `flex-shrink`, and `flex-basis` to control how items resize
- Build several genuinely common real layouts (navbar, centering, equal-height columns) using only Flexbox

Why Flexbox exists

Yesterday's tools (`display`, `position`) can position individual elements, but they're genuinely painful for a very common need: **arranging a row (or column) of items, evenly spaced or aligned, that resize sensibly as the available space changes**. Before Flexbox existed, developers relied on hacky workarounds (misusing `float`, or the table-for-layout anti-pattern from Day 2) to achieve what should be simple layouts. **Flexbox** ("Flexible Box Layout") was introduced specifically to solve this class of problem well, and it's used constantly in real, modern websites.

The core mental model: container and items

Flexbox always involves exactly two roles: a **flex container** (the parent, where you turn Flexbox on) and its **flex items** (its direct children, which then automatically follow Flexbox's layout rules). You turn an element into a flex container with one declaration:

```
.container {  
  display: flex;  
}
```

The instant you do this, every *direct child* of `.container` automatically becomes a flex item and starts following Flexbox's layout rules — no changes needed on the children themselves for this basic behavior. The single most immediately visible effect: **flex items line up in a row, side by side, automatically** — no more `display: inline-block` or float hacks needed just to get things sitting next to each other.

Main axis vs. cross axis — the idea that unlocks everything else

This is the concept that makes every other Flexbox property make sense, so read it carefully. A flex container has two perpendicular axes:

- The **main axis** — the primary direction items are laid out along. By default, this is **horizontal** (left to right).
- The **cross axis** — perpendicular to the main axis. By default, this is **vertical**.

```
.container {
  display: flex;
  flex-direction: row;      /* the DEFAULT -- main axis is horizontal, items go left to right */
  /* flex-direction: column; -- this would make the main axis VERTICAL instead, items stack top to bottom */
}
```

Every alignment property in Flexbox is defined *relative to whichever axis is currently the main axis* — this is exactly why **flex-direction** is the very first thing to understand, since it determines what "main axis" and "cross axis" actually mean for the rest of your rules.

Aligning along the main axis: `justify-content`

```
.container {
  display: flex;
  justify-content: flex-start; /* DEFAULT -- items packed at the start of the main axis */
  justify-content: flex-end;   /* items packed at the end */
  justify-content: center;     /* items packed in the center */
  justify-content: space-between; /* first item at start, last at end, even gaps between the items */
  justify-content: space-around; /* even gaps around EACH item (including half-gaps at both ends) */
  justify-content: space-evenly; /* perfectly even gaps everywhere, including both ends */
}
```

space-between is, in practice, the single most common choice for things like a navigation bar with a logo on the left and links on the right — it's worth trying all five values against the same row of items in today's exercise so the differences are visually obvious rather than abstract.

Aligning along the cross axis: `align-items`

```
.container {
  display: flex;
  align-items: stretch; /* DEFAULT -- items stretch to fill the container's cross-axis size */
  align-items: flex-start; /* items align to the start of the cross axis */
  align-items: flex-end;   /* items align to the end of the cross axis */
  align-items: center;     /* items centered on the cross axis */
}
```

This is, genuinely, the most common way to vertically center something in CSS — a task that was notoriously awkward before Flexbox existed: **display: flex;** plus **align-items: center;** on the parent centers its children vertically with almost no extra effort. **align-self** (set on an individual flex item, not the container) lets you override **align-items** for just that one specific item, when everything else should align one way but a single item needs to be different.

Controlling how items grow and shrink: `flex-grow`, `flex-shrink`, `flex-basis`

By default, flex items are only as big as their content requires, and any leftover space in the container is simply left empty. Often, you want items to expand and fill that leftover space instead:

```
.sidebar {
  flex-grow: 0; /* DEFAULT -- don't grow to fill extra space */
}
.main-content {
  flex-grow: 1; /* grow to fill the container's leftover space */
}
```

```

        flex-grow: 1;      /* DO grow to consume any leftover space in the container */
    }

```

If several items all have `flex-grow: 1`, the leftover space is split evenly among them; if one item has `flex-grow: 2` and others have `flex-grow: 1`, that item gets twice as large a share of the leftover space as each of the others — `flex-grow` values describe a *ratio*, not an absolute size.

`flex-shrink` works in reverse: it controls whether an item is allowed to shrink *below* its natural size when the container is too small to fit everything (default `1`, meaning "yes, allowed to shrink" — set to `0` to prevent an item from ever shrinking, even if it causes overflow). `flex-basis` sets an item's initial, "ideal" size along the main axis, before any growing or shrinking is applied — often used instead of `width` for flex items, since it's specifically main-axis-aware.

The shorthand you'll see constantly in real code combines all three: `flex: 1;` is shorthand for `flex-grow: 1; flex-shrink: 1; flex-basis: 0%;` — an extremely common pattern meaning "this item should grow to fill available space, ignoring its content's natural size as a starting point."

`gap` — spacing between flex items, the modern easy way

```

.container {
    display: flex;
    gap: 20px;      /* 20px of space between EVERY adjacent pair of flex items, but not around the outer edges */
}

```

`gap` is dramatically simpler than the older approach of adding `margin` to individual items (which requires careful handling of the first/last item to avoid unwanted extra space at the very edges) — always reach for `gap` on a flex or grid container instead.

Three real layouts, built entirely with what you now know

A navbar (logo left, links right, vertically centered):

```

.navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
}

```

Perfect centering (one item, centered both horizontally and vertically, in its parent):

```

.centered-container {
    display: flex;
    justify-content: center;
    align-items: center;
    height: 300px;
}

```

Equal-height columns (a notoriously awkward problem before Flexbox — with `align-items: stretch`, the default, every column automatically matches the height of the tallest one, with zero extra code):

```

.columns {
    display: flex;
    gap: 20px;
    /* align-items: stretch is the default -- no need to write it explicitly */
}

```

```
}  
.column {  
  flex: 1;    /* each column grows equally to fill the available width */  
}
```

Exercises

Open `starter.html` and `starter.css`, follow the `<!-- TODO --> /* TODO */` markers, and check your work against `CHECKLIST.md`.